Openjpeg-2.1.2 Heap Buffer Overflow Vulnerability due to insufficient check

Overview

I have found a vulnerability in openjpeg-2.1.2 (an open-source JPEG 2000 codec written in C language) using AFL (http://lcamtuf.coredump.cx/afl/). The vulnerability is an heap buffer overflow vulnerability, which can cause out-of-bound read. The vulnerability exists in code responsible for decompressing the input image. The vulnerability is caused by insufficient check: the code have made sure the parameters of RED channel, GREEN channel and BLUE channel are the same, however, does not do it for ALPHA channel, which causes the length of alpha buffer is less than the expected access length. The vulnerability can cause Denial-of-Service and Information Disclosure and I am not sure whether it can cause RCE.

Software and Environments

Software: openjpeg-2.1.2 (http://www.openjpeg.org/)
(download link: https://github.com/uclouvain/openjpeg/archive/v2.1.2.tar.gz)

Operating System: Ubuntu 16.04.1 i686 LTS
lsb_release –a :

```
Distributor ID: Ubuntu
Description:    Ubuntu 16.04.1 LTS
Release:  16.04
Codename:     xenial
```

uname –a :

```
Linux ubuntu 4.4.0-43-generic #63-Ubuntu SMP Wed Oct 12 13:50:36 UTC 2016 i686 i686 i686 GNU/Linux
```

Compilers: GCC + CLANG + rr
gcc --version :

```
gcc (Ubuntu 5.4.0-6ubuntu1~16.04.2) 5.4.0 20160609
Copyright (C) 2015 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.    There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

clang --version:

rr --version:

```
rr version 4.4.0
```

```
clang version 3.8.0-2ubuntu4 (tags/RELEASE_380/final)
Target: i686-pc-linux-gnu
Thread model: posix
InstalledDir: /usr/bin
```

Reproduction

GCC + Debug:

```
cd /path of openjpeg source code/
mkdir build-gcc-debug
cd build-gcc-debug
cmake --DCMAKE_BUILD_TYPE=Debug ..
make
cd bin & ./opj_decompress –i PoC –o a.ppm
```

Clang + ASAN:

```
cd /path of openjpeg source code/
mkdir build-clang-asan
cd build-clang-asan
CC=clang CXX=clang++ CFLAGS=” -fsanitize=address” CXXFLAGS=” -fsanitize=address” cmake ..
make
cd bin & ./opj_decompress –i PoC –o a.ppm
```

Exception

Decompress the PoC file with command `./opj_decompress -i PoC –o a.ppm`, where 'opj_decompress' is compiled by clang with AddressSanitizer switch:

```
[INFO] Stream reached its end !
=================================================================
==2268==ERROR: AddressSanitizer: heap-buffer-overflow on address 0xb2610880 at pc 0x0815764b bp 0xbf9da7a8 sp 0xbf9da79c
READ of size 4 at 0xb2610880 thread T0
    #0 0x815764a  (/home/fire/bing/afl/libraries/openjpeg-2.1.2/build-clang-asan/bin/opj_decompress+0x815764a)
    #1 0x813b969  (/home/fire/bing/afl/libraries/openjpeg-2.1.2/build-clang-asan/bin/opj_decompress+0x813b969)
    #2 0xb73e6636  (/lib/i386-linux-gnu/libc.so.6+0x18636)
    #3 0x805f337  (/home/fire/bing/afl/libraries/openjpeg-2.1.2/build-clang-asan/bin/opj_decompress+0x805f337)

0xb2610880 is located 0 bytes to the right of 65664-byte region [0xb2600800,0xb2610880)
allocated by thread T0 here:
    #0 0x8103804  (/home/fire/bing/afl/libraries/openjpeg-2.1.2/build-clang-asan/bin/opj_decompress+0x8103804)
    #1 0xb774ee46  (/home/fire/bing/afl/libraries/openjpeg-2.1.2/build-clang-asan/bin/libopenjp2.so.7+0x118e46)
    #2 0xb76ae89f  (/home/fire/bing/afl/libraries/openjpeg-2.1.2/build-clang-asan/bin/libopenjp2.so.7+0x7889f)
    #3 0xb76ae100  (/home/fire/bing/afl/libraries/openjpeg-2.1.2/build-clang-asan/bin/libopenjp2.so.7+0x78100)
    #4 0xb766a294  (/home/fire/bing/afl/libraries/openjpeg-2.1.2/build-clang-asan/bin/libopenjp2.so.7+0x34294)
    #5 0xb7681f9e  (/home/fire/bing/afl/libraries/openjpeg-2.1.2/build-clang-asan/bin/libopenjp2.so.7+0x4bf9e)
    #6 0xb76c8e50  (/home/fire/bing/afl/libraries/openjpeg-2.1.2/build-clang-asan/bin/libopenjp2.so.7+0x92e50)
    #7 0xb76ebe43  (/home/fire/bing/afl/libraries/openjpeg-2.1.2/build-clang-asan/bin/libopenjp2.so.7+0xb5e43)
    #8 0x8139872  (/home/fire/bing/afl/libraries/openjpeg-2.1.2/build-clang-asan/bin/opj_decompress+0x8139872)
    #9 0xb73e6636  (/lib/i386-linux-gnu/libc.so.6+0x18636)
```

```
SUMMARY: AddressSanitizer: heap-buffer-overflow (/home/fire/bing/afl/libraries/openjpeg-2.1.2/build-clang-asan/bin/opj_decompress+0x815764a)
Shadow bytes around the buggy address:
  0x364c20c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x364c20d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x364c20e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x364c20f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x364c2100: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x364c2110:[fa]fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0x364c2120: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0x364c2130: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0x364c2140: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0x364c2150: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0x364c2160: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
Shadow byte legend (one shadow byte represents 8 application bytes):
  Addressable:           00
  Partially addressable: 01 02 03 04 05 06 07
  Heap left redzone:       fa
  Heap right redzone:      fb
  Freed heap region:       fd
  Stack left redzone:      f1
  Stack mid redzone:       f2
  Stack right redzone:     f3
  Stack partial redzone:   f4
  Stack after return:      f5
  Stack use after scope:   f8
  Global redzone:          f9
  Global init order:       f6
  Poisoned by user:        f7
  Container overflow:      fc
  Array cookie:            ac
  Intra object redzone:    bb
  ASan internal:           fe
  Left alloca redzone:     ca
  Right alloca redzone:    cb
==2268==ABORTING
```

As above shown, this is a heap buffer overflow vulnerability.

Analysis

The crash happens due to an invalid memory read at line "openjpeg-2.1.2/src/bin/jp2/convert.c: 1885":

```
[INFO] Stream reached its end !

Program received signal SIGSEGV, Segmentation fault.
0x080518f5 in imagetopnm (image=0x84d78a8, outfile=0xbf8f7504 "a.ppm", force_split=0)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/bin/jp2/convert.c:1885
1885                    v = *alpha++;
(rr) p *alpha
Cannot access memory at address 0x8892004
(rr) bt
#0  0x080518f5 in imagetopnm (image=0x84d78a8, outfile=0xbf8f7504 "a.ppm", force_split=0)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/bin/jp2/convert.c:1885
#1  0x0804c5fe in main (argc=5, argv=0xbf8f9604) at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/bin/jp2/opj_decompress.c:1467
(rr)
```

Analyzing the source code, I find the crash code lies in a loop body:

```
1829            for(i = 0; i < wr * hr; ++i)
1830            {                                                    covert.c
1831                if(two)
1832                {
1833                    v = *red + adjustR; ++red;
1834                    if(v > 65535) v = 65535; else if(v < 0) v = 0;
1865                }   /* if(two) */
1866
1883                if(has_alpha)
1884                {
1885                    v = *alpha++;
1886                    if(v > 255) v = 255; else if(v < 0) v = 0;
1887
1888                    fprintf(fdest, "%c", (unsigned char)v);
1889                }
1890            }   /* for(i */
```

I guess the invalid memory read is probably an out-of-bound read. Check the value of '**i**' when crashing and the maximum loop times "**wr * hr**":

```
(rr) p/x i
$1 = 0x8024
(rr) p/x wr
$2 = 0x4020
(rr) p/x hr
$3 = 0x20
(rr) p/x wr*hr
$4 = 0x80400
(rr)
```

From above picture, I guess this is probably an out-of-bound read. To confirm it, I need to make sure the length of array "**alpha**" is less than "**wr*hr**". For this, I first need to the source of "**alpha**": Set breakpoint at the crash line and make the program break just before the crash happening (In my debugging, the breakpoint should be hit **32805** times, the number may be different in other debuggings)

```
Breakpoint 1, imagetopnm (image=0x84d78a8, outfile=0xbf8f7504 "a.ppm", force_split=0)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/bin/jp2/convert.c:1885
1885                    v = *alpha++;
(rr) p *alpha
Cannot access memory at address 0x8892000
(rr) info break 1
Num     Type           Disp Enb Address    What
1       breakpoint     keep y   0x080518ec in imagetopnm at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/bin/jp2/convert.c:1885
        breakpoint already hit 32805 times
(rr)
```

Viewing the source code, I can easily find the source of "**alpha**" comes from "**image->comps[ncomp - 1].data**":

```
1803            if(has_alpha)
1804            {                                                    covert.c
1805                const char *tt = (triple?"RGB_ALPHA":"GRAYSCALE_ALPHA");
1806
1807                fprintf(fdest, "P7\n# OpenJPEG-%s\nWIDTH %d\nHEIGHT %d\nDEPTH %u\n"
1808                        "MAXVAL %d\nTUPLTYPE %s\nENDHDR\n", opj_version(),
1809                        wr, hr, ncomp, max, tt);
1810                alpha = image->comps[ncomp - 1].data;
1811                adjustA = (image->comps[ncomp - 1].sgnd ?
1812                           1 << (image->comps[ncomp - 1].prec - 1) : 0);
1813            }
```

Set breakpoint at **line 1810:convert.c** and reverse-continue:

```
(rr) reverse-continue
Continuing.

Breakpoint 3, imagetopnm (image=0x84d78a8, outfile=0xbf8f7504 "a.ppm", force_split=0)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/bin/jp2/convert.c:1810
1810                alpha = image->comps[ncomp - 1].data;
(rr) p/x ncomp
$5 = 0x4
(rr) p image->comps[3]
$6 = {dx = 1, dy = 12[illegible], w = 1641[illegible], h = 1, x0 = 0, y0 = 0, prec = 1, bpp = 0, sgnd = 0, resno_decoded = 5, factor = 0, data = 0x8871f70,
  alpha = 1}
(rr) p image->comps[2]
$7 = {dx = 1, dy = 1, w = 16416, h = 32, x0 = 0, y0 = 0, prec = 5, bpp = 0, sgnd = 0, resno_decoded = 5, factor = 0, data = 0xb6a37008,
  alpha = 0}
(rr) p image->comps[1]
$8 = {dx = 1, dy = 1, w = 16416, h = 32, x0 = 0, y0 = 0, prec = 5, bpp = 0, sgnd = 0, resno_decoded = 5, factor = 0, data = 0xb6c39008,
  alpha = 0}
(rr) p image->comps[0]
$9 = {dx = 1, dy = 1, w = 16416, h = 32, x0 = 0, y0 = 0, prec = 5, bpp = 0, sgnd = 0, resno_decoded = 5, factor = 0, data = 0xb6e3b008,
  alpha = 0}
(rr)
```

The breakpoint is hit; "**alpha**" really comes from "**image->comps[3].data**". Next, I need to know the length of array "**image->comps[3].data**".

Set watchpoint for "**image->comps[3].data**", Before reverse-continue, I can easily know "**wr**" comes from "**image->comps[0].w**" and "**hr**" comes from "**image->comps[0].h**", which is confirmed in the following picture:

```
(rr) watch -l image->comps[3].data
Hardware watchpoint 4: -location image->comps[3].data
(rr) break /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/bin/jp2/convert.c:1791
Breakpoint 5 at 0x80514a8: file /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/bin/jp2/convert.c, line 1791.
(rr) reverse-continue
Continuing.

Breakpoint 5, imagetopnm (image=0x84d78a8, outfile=0xbf8f7504 "a.ppm", force_split=0)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/bin/jp2/convert.c:1791
1791            wr = (int)image->comps[0].w; hr = (int)image->comps[0].h;
(rr) p/x image->comps[0]
$10 = {dx = 0x1, dy = 0x1, w = 0x4020, h = 0x20, x0 = 0x0, y0 = 0x0, prec = 0x5, bpp = 0x0, sgnd = 0x0, resno_decoded = 0x5, factor = 0x0,
  data = 0xb6e3b008, alpha = 0x0}
(rr) reverse-continue
Continuing.

Hardware watchpoint 4: -location image->comps[3].data

Old value = (OPJ_INT32 *) 0x8871f70
New value = (OPJ_INT32 *) 0x0
0xb7761aa6 in opj_j2k_decode (p_j2k=0x84d52b0, p_stream=0x84d4180, p_image=0x84d78a8, p_manager=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/j2k.c:9978
9978                    p_image->comps[compno].data = p_j2k->m_output_image->comps[compno].data;
(rr) p/x compno
$11 = 0x3
(rr)
```

From above, I see "**image->comps[3].data**" comes from "**p_j2k->m_output_image->comps [3].data**". Same with the above, set watchpoint for the latter and reverse-continue:

```
(rr) watch -l p_j2k->m_output_image->comps[3].data
Hardware watchpoint 6: -location p_j2k->m_output_image->comps[3].data
(rr) reverse-continue
Continuing.

Hardware watchpoint 6: -location p_j2k->m_output_image->comps[3].data

Old value = (OPJ_INT32 *) 0x8871f70
New value = (OPJ_INT32 *) 0x0
0xb775d8db in opj_j2k_update_image_data (p_tcd=0x84d7868, p_data=0x8871f48 "", p_output_image=0x84d78d0)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/j2k.c:8231
8231                    l_img_comp_dest->data = (OPJ_INT32*) opj_calloc(l_width * l_height, sizeof(OPJ_INT32));
(rr)
```

From the above picture, I know "**p_j2k->m_output_image->comps [3].data**" is allocated by some alloc function (here is **opj_calloc**). The source code of "**opj_calloc**" function:

opj_malloc.c

```
197  void * opj_calloc(size_t num, size_t size)
198  {
199    if (num == 0 || size == 0) {
200      /* prevent implementation defined behavior of realloc */
201      return NULL;
202    }
203    return calloc(num, size);
204  }
205
```

Now, I need to know the requested allocation size:

```
Hardware watchpoint 6: -location p_j2k->m_output_image->comps[3].data

Old value = (OPJ_INT32 *) 0x8871f70
New value = (OPJ_INT32 *) 0x0
0xb775d8db in opj_j2k_update_image_data (p_tcd=0x84d7868, p_data=0x8871f48 "", p_output_image=0x84d78d0)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/j2k.c:8231
8231                    l_img_comp_dest->data = (OPJ_INT32*) opj_calloc(l_width * l_height, sizeof(OPJ_INT32));
(rr) p/x l_width
$12 = 0x4020
(rr) p/x l_height
$13 = 0x1
(rr) p/x sizeof(OPJ_INT32)
$14 = 0x4
(rr) p/x l_width*l_height*sizeof(OPJ_INT32)
$15 = 0x10080
(rr)
```

From above, the allocated byte size is **(0x4020 * 0x1 * 0x4)**, i.e. the length of **int** array "**alpha**" is **(0x4020 * 0x01)**, which is less than the above loop times "**wr * hr = (0x4020 * 0x20)**". So, this is really an out-of-bound read vulnerability.

Next, I need to know why the actual requested size is different from the expected value and how to modify a normal input image to trigger this.

From the code shown as the following, I can know "l_width" and "l_height" come from "**p_output_image->comps[i].w**" and "**p_out_image->comps[i].h**"; "i" is some value between **0** and **l_image_src->numcomps**.

j2k.c

```
8217
8218            l_img_comp_dest = p_output_image->comps;
8219
8220            for (i=0; i<l_image_src->numcomps; i++) {
8221
8222                    /* Allocate output component buffer if necessary */
8223                    if (!l_img_comp_dest->data) {
8224                            OPJ_SIZE_T l_width = l_img_comp_dest->w;
8225                            OPJ_SIZE_T l_height = l_img_comp_dest->h;
8226
8227                            if ((l_height == 0U) || (l_width > (SIZE_MAX / l_height))) {
8228                                    /* would overflow */
8229                                    return OPJ_FALSE;
8230                            }
8231                            l_img_comp_dest->data = (OPJ_INT32*) opj_calloc(l_width * l_height, sizeof(OPJ_INT32));
8232                            if (! l_img_comp_dest->data) {
8233                                    return OPJ_FALSE;
8234                            }
8235                    }
```

In the debugging, check the value of "**i**" and "**l_img_comp_dest**"; then set breakpoint at line **8218: j2k.c** and reverse-continue:

```
(rr) p/x *l_img_comp_dest
$16 = {dx = 0x1, dy = 0x81, w = 0x4020, h = 0x1, x0 = 0x0, y0 = 0x0, prec = 0x1, bpp = 0x0, sgnd = 0x0, resno_decoded = 0x0, factor = 0x0,
  data = 0x0, alpha = 0x0}
(rr) p/x i
$17 = 0x3
(rr) p/x p_output_image_comps[3]
No symbol "p_output_image_comps" in current context.
(rr) break /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/j2k.c:8218
Breakpoint 7 at 0xb775d86b: file /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/j2k.c, line 8218.
(rr) reverse-continue
Continuing.

Breakpoint 7, opj_j2k_update_image_data (p_tcd=0x84d7868, p_data=0x8871348 '\037' <repeats 200 times>..., p_output_image=0x84d78d0)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/j2k.c:8218
8218            l_img_comp_dest = p_output_image->comps;
(rr) p/x p_output_image->comps[3]
$18 = {dx = 0x1, dy = 0x81, w = 0x4020, h = 0x1, x0 = 0x0, y0 = 0x0, prec = 0x1, bpp = 0x0, sgnd = 0x0, resno_decoded = 0x0, factor = 0x0,
  data = 0x0, alpha = 0x0}
(rr) p/x p_output_image->comps[2]
$19 = {dx = 0x1, dy = 0x1, w = 0x4020, h = 0x20, x0 = 0x0, y0 = 0x0, prec = 0x5, bpp = 0x0, sgnd = 0x0, resno_decoded = 0x0, factor = 0x0,
  data = 0x0, alpha = 0x0}
(rr) p/x p_output_image->comps[1]
$20 = {dx = 0x1, dy = 0x1, w = 0x4020, h = 0x20, x0 = 0x0, y0 = 0x0, prec = 0x5, bpp = 0x0, sgnd = 0x0, resno_decoded = 0x0, factor = 0x0,
  data = 0x0, alpha = 0x0}
(rr) p/x p_output_image->comps[0]
$21 = {dx = 0x1, dy = 0x1, w = 0x4020, h = 0x20, x0 = 0x0, y0 = 0x0, prec = 0x5, bpp = 0x0, sgnd = 0x0, resno_decoded = 0x0, factor = 0x0,
  data = 0x0, alpha = 0x0}
(rr)
```

"**i**" is **3**; the requested allocated size is equal to "**p_output_image->comps[3].w**" * "**p_output_image->comps[3].h**". From prior analysis, the total loop times when crashing is "**image->comps[0].w**" (0x4020) * "**image->comps[0].h**" (0x20), which is same with **p_output_image->comps[0].w(h)**. Viewing the 4 elements of array "**comps**", I guess the cause of

the out-of-bound read is probably that “**p_output_image->comps[3].h**” is **0x01** instead of **0x20.**

Next I need to know why “**p_output_image->comps[3].h**” is **0x01**, i.e. find the source of it.

Set watch point for “**p_output_image->comps[3].h**” and reverse-continue.

```
(rr) watch -l p_output_image->comps[3].h
Hardware watchpoint 8: -location p_output_image->comps[3].h
(rr) reverse-continue
Continuing.

Hardware watchpoint 8: -location p_output_image->comps[3].h

Old value = 1
New value = 0
__memcpy_sse2_unaligned () at ../sysdeps/i386/i686/multiarch/memcpy-sse2-unaligned.S:485
485     ../sysdeps/i386/i686/multiarch/memcpy-sse2-unaligned.S: No such file or directory.
(rr) bt
#0  __memcpy_sse2_unaligned () at ../sysdeps/i386/i686/multiarch/memcpy-sse2-unaligned.S:485
#1  0xb774e3e1 in opj_copy_image_header (p_image_src=0x84d78a8, p_image_dest=0x84d78d0)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/image.c:182
#2  0xb7761a00 in opj_j2k_decode (p_j2k=0x84d52b0, p_stream=0x84d4180, p_image=0x84d78a8, p_manager=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/j2k.c:9963
#3  0xb77664cf in opj_jp2_decode (jp2=0x84d5228, p_stream=0x84d4180, p_image=0x84d78a8, p_manager=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/jp2.c:1502
#4  0xb776b1f3 in opj_decode (p_codec=0x84d51d0, p_stream=0x84d4180, p_image=0x84d78a8)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/openjpeg.c:412
#5  0x0804c14d in main (argc=5, argv=0xbf8f9604) at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/bin/jp2/opj_decompress.c:1332
(rr)
```

“**p_output_image->comps[3].h**” is copied from some source buffer.

image.c

```
181         for (compno=0; compno < p_image_dest->numcomps; compno++){
182             memcpy( &(p_image_dest->comps[compno]),
183                     &(p_image_src->comps[compno]),
184                     sizeof(opj_image_comp_t));
185             p_image_dest->comps[compno].data = NULL;
186         }
187
```

Similar with above operation, continue reverse-tracking.

```
(rr) break /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/image.c:182
Breakpoint 9 at 0xb774e3c7: file /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/image.c, line 182.
(rr) reverse-continue
Continuing.

Breakpoint 9, opj_copy_image_header (p_image_src=0x84d78a8, p_image_dest=0x84d78d0)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/image.c:182
182                     memcpy( &(p_image_dest->comps[compno]),
(rr) p/x compno
$22 = 0x3
(rr) p/x p_image_src->comps[3]
$23 = {dx = 0x1, dy = 0x81, w = 0x4020, h = 0x1, x0 = 0x0, y0 = 0x0, prec = 0x1, bpp = 0x0, sgnd = 0x0, resno_decoded = 0x0, factor = 0x0,
  data = 0x0, alpha = 0x0}
(rr) watch -l p_image_src->comps[3].h
Hardware watchpoint 10: -location p_image_src->comps[3].h
(rr) disable 9
(rr) reverse-continue
Continuing.

Hardware watchpoint 8: -location p_output_image->comps[3].h

Old value = 0
New value = <unreadable>
0xb7790b25 in ?? () from /usr/bin/../lib/librrpreload.so
```

```
(rr) reverse-continue
Continuing.

Hardware watchpoint 10: -location p_image_src->comps[3].h

Old value = 1
New value = 0
__memcpy_sse2_unaligned () at ../sysdeps/i386/i686/multiarch/memcpy-sse2-unaligned.S:485
485     ../sysdeps/i386/i686/multiarch/memcpy-sse2-unaligned.S: No such file or directory.
(rr) bt
#0  __memcpy_sse2_unaligned () at ../sysdeps/i386/i686/multiarch/memcpy-sse2-unaligned.S:485
#1  0xb774e3e1 in opj_copy_image_header (p_image_src=0x84d74f8, p_image_dest=0x84d78a8)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/image.c:182
#2  0xb775a747 in opj_j2k_read_header (p_stream=0x84d4180, p_j2k=0x84d52b0, p_image=0xbf8f4474, p_manager=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/j2k.c:6844
#3  0xb7768c53 in opj_jp2_read_header (p_stream=0x84d4180, jp2=0x84d5228, p_image=0xbf8f4474, p_manager=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/jp2.c:2666
#4  0xb776b18f in opj_read_header (p_stream=0x84d4180, p_codec=0x84d51d0, p_image=0xbf8f4474)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/openjpeg.c:391
#5  0x0804c038 in main (argc=5, argv=0xbf8f9604) at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/bin/jp2/opj_decompress.c:1312
(rr) enable 9
(rr) reverse-continue
Continuing.

Breakpoint 9, opj_copy_image_header (p_image_src=0x84d74f8, p_image_dest=0x84d78a8)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/image.c:182
182                     memcpy( &(p_image_dest->comps[compno]),
(rr) p/x compno
$24 = 0x3
(rr) p/x p_image_src->comps[3]
$25 = {dx = 0x1, dy = 0x81, w = 0x4020, h = 0x1, x0 = 0x0, y0 = 0x0, prec = 0x1, bpp = 0x0, sgnd = 0x0, resno_decoded = 0x0, factor = 0x0,
  data = 0x0, alpha = 0x0}
(rr) disable 8 10
```

```
(rr) watch -l p_image_src->comps[3].h
Hardware watchpoint 11: -location p_image_src->comps[3].h
(rr) reverse-continue
Continuing.

Breakpoint 9, opj_copy_image_header (p_image_src=0x84d74f8, p_image_dest=0x84d78a8)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/image.c:182
182                     memcpy( &(p_image_dest->comps[compno]),
(rr) disable 9
(rr) reverse-continue
Continuing.

Hardware watchpoint 11: -location p_image_src->comps[3].h

Old value = 1
New value = 0
0xb774e23d in opj_image_comp_header_update (p_image_header=0x84d74f8, p_cp=0x84d5304)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/image.c:132
132                     l_img_comp->h = l_height;
(rr) p/x i
$26 = 0x3
(rr)
```

I can backward track to line **132: image.c** where the code is shown as the following:

image.c

```
116     l_x0 = opj_uint_max(p_cp->tx0 , p_image_header->x0);
117     l_y0 = opj_uint_max(p_cp->ty0 , p_image_header->y0);
118     l_x1 = p_cp->tx0 + (p_cp->tw - 1U) * p_cp->tdx; /* validity of p_cp members used here checked in opj_j2k_read_siz.
119     l_y1 = p_cp->ty0 + (p_cp->th - 1U) * p_cp->tdy; /* can't overflow */
120     l_x1 = opj_uint_min(opj_uint_adds(l_x1, p_cp->tdx), p_image_header->x1); /* use add saturated to prevent overflow *
121     l_y1 = opj_uint_min(opj_uint_adds(l_y1, p_cp->tdy), p_image_header->y1); /* use add saturated to prevent overflow *
122
123     l_img_comp = p_image_header->comps;
124     for (i = 0; i < p_image_header->numcomps; ++i) {
125         l_comp_x0 = opj_uint_ceildiv(l_x0, l_img_comp->dx);
126         l_comp_y0 = opj_uint_ceildiv(l_y0, l_img_comp->dy);
127         l_comp_x1 = opj_uint_ceildiv(l_x1, l_img_comp->dx);
128         l_comp_y1 = opj_uint_ceildiv(l_y1, l_img_comp->dy);
129         l_width   = opj_uint_ceildivpow2(l_comp_x1 - l_comp_x0, l_img_comp->factor);
130         l_height  = opj_uint_ceildivpow2(l_comp_y1 - l_comp_y0, l_img_comp->factor);
131         l_img_comp->w = l_width;
132         l_img_comp->h = l_height;
133         l_img_comp->x0 = l_comp_x0;
134         l_img_comp->y0 = l_comp_y0;
135         ++l_img_comp;
136     }
```

(l_y0+l_img_comp->dy -1) / l_img_comp->dy

(l_y1 + l_img_comp->dy -1)/l_img_comp->dy

(l_comp_y1 -l_comp_y0)/2^(l_img_comp->factor)

From above code:

l_y0 = max{p_cp->ty0, p_image_header->y0}
l_y1 = min{p_image_header->y1, (p_cp->ty0 + (p_cp->th -1) * p_cp->tdy) + p_cp->tdy}
l_comp_y0 = (l_y0+p_image_header->comps[i].dy - 1) / p_image_header->comps[i].dy
l_comp_y1 = (l_y1+p_image_header->comps[i].dy - 1) / p_image_header->comps[i].dy
l_height = (l_comp_y1 – l_comp_y0) / 2^(p_image_header->comps[i].factor)
(0 <= i < p_image_header->numcomps)

I find "**comps[i].h**" is only related with "**comps[i].dy**" and "**comps[i].factor**"; for each comps[i], other parameters are the same. In the following picture, I find the only differences of each field among the **comps[i]**s are the values of field "**dy**" and "**prec**". "**prec**" is not related with the computation of "**comps[i].h**". Further, I find "**comps[i].dy**" is just the element causing that "**comps[3].h**" is 0x01 instead of 0x20 which is the value of "**comps[0\1\2]**".

```
(rr) break /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/image.c:123
Breakpoint 12 at 0xb774e183: file /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/image.c, line 123.
(rr) reverse-continue
Continuing.

Breakpoint 12, opj_image_comp_header_update (p_image_header=0x84d74f8, p_cp=0x84d5304)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/image.c:123
123             l_img_comp = p_image_header->comps;
(rr) p/x p_image_header->comps[3]
$29 = {dx = 0x1, dy = 0x81, w = 0x0, h = 0x0, x0 = 0x0, y0 = 0x0, prec = 0x1, bpp = 0x0, sgnd = 0x0, resno_decoded = 0x0, factor = 0x0,
  data = 0x0, alpha = 0x0}
(rr) p/x p_image_header->comps[2]
$30 = {dx = 0x1, dy = 0x1, w = 0x0, h = 0x0, x0 = 0x0, y0 = 0x0, prec = 0x5, bpp = 0x0, sgnd = 0x0, resno_decoded = 0x0, factor = 0x0,
  data = 0x0, alpha = 0x0}
(rr) p/x p_image_header->comps[1]
$31 = {dx = 0x1, dy = 0x1, w = 0x0, h = 0x0, x0 = 0x0, y0 = 0x0, prec = 0x5, bpp = 0x0, sgnd = 0x0, resno_decoded = 0x0, factor = 0x0,
  data = 0x0, alpha = 0x0}
(rr) p/x p_image_header->comps[0]
$32 = {dx = 0x1, dy = 0x1, w = 0x0, h = 0x0, x0 = 0x0, y0 = 0x0, prec = 0x5, bpp = 0x0, sgnd = 0x0, resno_decoded = 0x0, factor = 0x0,
  data = 0x0, alpha = 0x0}
(rr)
```

Next, I need to know why "**comps[3].dy**" is 0x81 instead of 0x01.

Continue reverse tracking "**comps[3].dy**":

```
(rr) watch -l p_image_header->comps[3].dy
Hardware watchpoint 13: -location p_image_header->comps[3].dy
(rr) reverse-continue
Continuing.

Hardware watchpoint 13: -location p_image_header->comps[3].dy

Old value = 129
New value = 0
0xb77506ef in opj_j2k_read_siz (p_j2k=0x84d52b0, p_header_data=0x84d69c0 "", p_header_size=48, p_manager=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/j2k.c:2153
2153                    l_img_comp->dy = (OPJ_UINT32)tmp; /* should be between 1 and 255 */
(rr)
```

j2k.c

```
2139            l_img_comp = l_image->comps;
2140
2141            /* Read the component information */
2142            for (i = 0; i < l_image->numcomps; ++i){
2143                    OPJ_UINT32 tmp;
2144                    opj_read_bytes(p_header_data,&tmp,1);   /* Ssiz_i */
2145                    ++p_header_data;
2146                    l_img_comp->prec = (tmp & 0x7f) + 1;
2147                    l_img_comp->sgnd = tmp >> 7;
2148                    opj_read_bytes(p_header_data,&tmp,1);   /* XRsiz_i */
2149                    ++p_header_data;
2150                    l_img_comp->dx = (OPJ_UINT32)tmp; /* should be between 1 and 255 */
2151                    opj_read_bytes(p_header_data,&tmp,1);   /* YRsiz_i */
2152                    ++p_header_data;
2153                    l_img_comp->dy = (OPJ_UINT32)tmp; /* should be between 1 and 255 */
2154                    if( l_img_comp->dx < 1 || l_img_comp->dx > 255 ||
2155                        l_img_comp->dy < 1 || l_img_comp->dy > 255 ) {
2156                        opj_event_msg(p_manager, EVT_ERROR,
2157                                      "Invalid values for comp = %d : dx=%u dy=%u (should be between 1 and 255 according
2158                                      i, l_img_comp->dx, l_img_comp->dy);
2159                        return OPJ_FALSE;
2160                    }
```

"**l_img_comp->dy**" is read from buffer "**p_header_data**". Next, find where "p_header_data[-1]" comes from:

```
(rr) p/x i
$33 = 0x3
(rr) p/x p_header_data[-1]
$34 = 0x81
(rr) watch -l p_header_data[-1]
Hardware watchpoint 14: -location p_header_data[-1]
(rr) reverse-continue
Continuing.

Hardware watchpoint 14: -location p_header_data[-1]

Old value = 129 '\201'
New value = 0 '\000'
__memcpy_sse2_unaligned () at ../sysdeps/i386/i686/multiarch/memcpy-sse2-unaligned.S:486
486     ../sysdeps/i386/i686/multiarch/memcpy-sse2-unaligned.S: No such file or directory.
(rr) bt
#0  __memcpy_sse2_unaligned () at ../sysdeps/i386/i686/multiarch/memcpy-sse2-unaligned.S:486
#1  0xb774a457 in opj_stream_read_data (p_stream=0x84d4180, p_buffer=0x84d6990 "", p_size=48, p_event_mgr=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/cio.c:270
#2  0xb775b6b4 in opj_j2k_read_header_procedure (p_j2k=0x84d52b0, p_stream=0x84d4180, p_manager=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/j2k.c:7275
#3  0xb775b9f8 in opj_j2k_exec (p_j2k=0x84d52b0, p_procedure_list=0x84d7438, p_stream=0x84d4180, p_manager=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/j2k.c:7354
#4  0xb775a6ec in opj_j2k_read_header (p_stream=0x84d4180, p_j2k=0x84d52b0, p_image=0xbf8f4474, p_manager=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/j2k.c:6832
#5  0xb7768c53 in opj_jp2_read_header (p_stream=0x84d4180, jp2=0x84d5228, p_image=0xbf8f4474, p_manager=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/jp2.c:2666
#6  0xb776b18f in opj_read_header (p_stream=0x84d4180, p_codec=0x84d51d0, p_image=0xbf8f4474)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/openjpeg.c:391
#7  0x0804c038 in main (argc=5, argv=0xbf8f9604) at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/bin/jp2/opj_decompress.c:1312
(rr)
```

```
(rr) break /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/cio.c:270
Breakpoint 15 at 0xb774a442: file /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/cio.c, line 270.
(rr) reverse-continue
Continuing.

Breakpoint 15, opj_stream_read_data (p_stream=0x84d4180, p_buffer=0x84d6990 "", p_size=48, p_event_mgr=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/cio.c:270
270                     memcpy(p_buffer,p_stream->m_current_data,p_size);
(rr) p/x p_size
$35 = 0x30
(rr) x/48b p_stream->m_current_data
0xb7300091:     0x00    0x00    0x00    0x00    0x40    0x20    0x00    0x00
0xb7300099:     0x00    0x20    0x00    0x00    0x00    0x00    0x00    0x00
0xb73000a1:     0x00    0x00    0x00    0x00    0x00    0x20    0x00    0x00
0xb73000a9:     0x00    0x20    0x00    0x00    0x00    0x00    0x00    0x00
0xb73000b1:     0x00    0x00    0x00    0x04    0x04    0x01    0x01    0x04
0xb73000b9:     0x01    0x01    0x04    0x01    0x01    0x00    0x01    0x81
(rr) watch -l p_stream->m_current_data[47]
Hardware watchpoint 16: -location p_stream->m_current_data[47]
(rr) reverse-continue
Continuing.
Warning:
Could not insert hardware watchpoint 16.
Could not insert hardware breakpoints:
You may have requested too many hardware breakpoints/watchpoints.

Command aborted.
(rr) disable 14 15
```

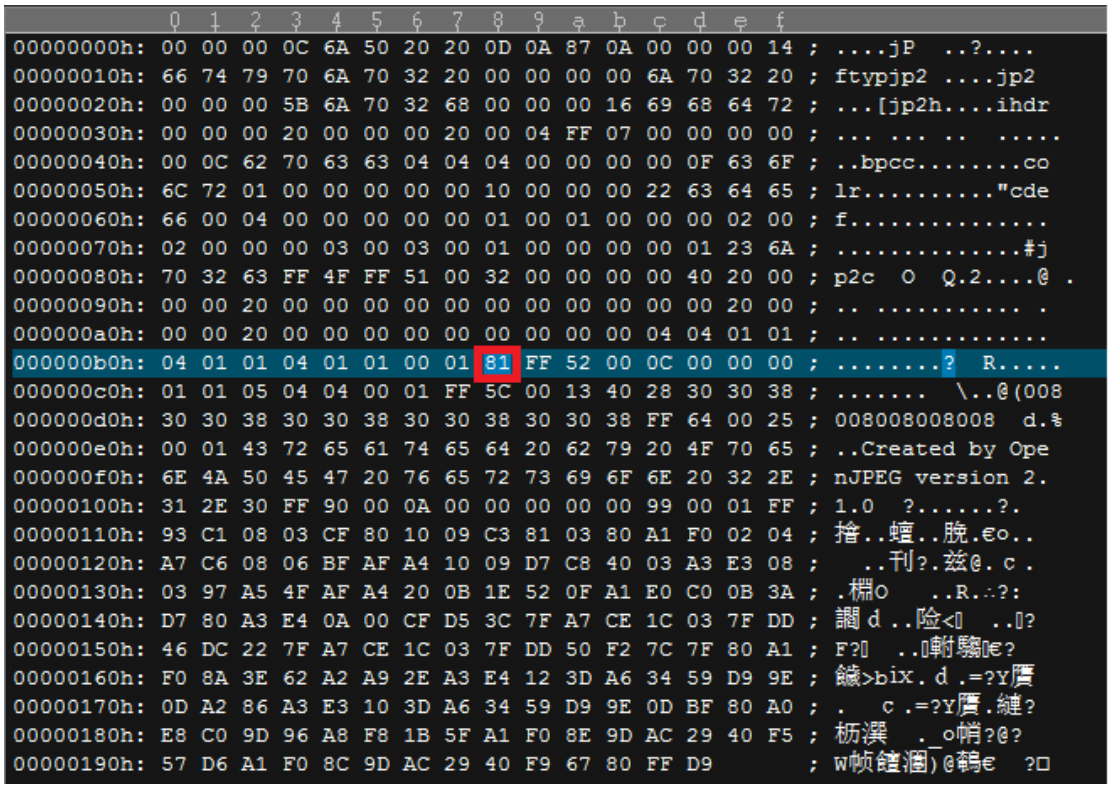
```
(rr) reverse-continue
Continuing.

Hardware watchpoint 16: -location p_stream->m_current_data[47]

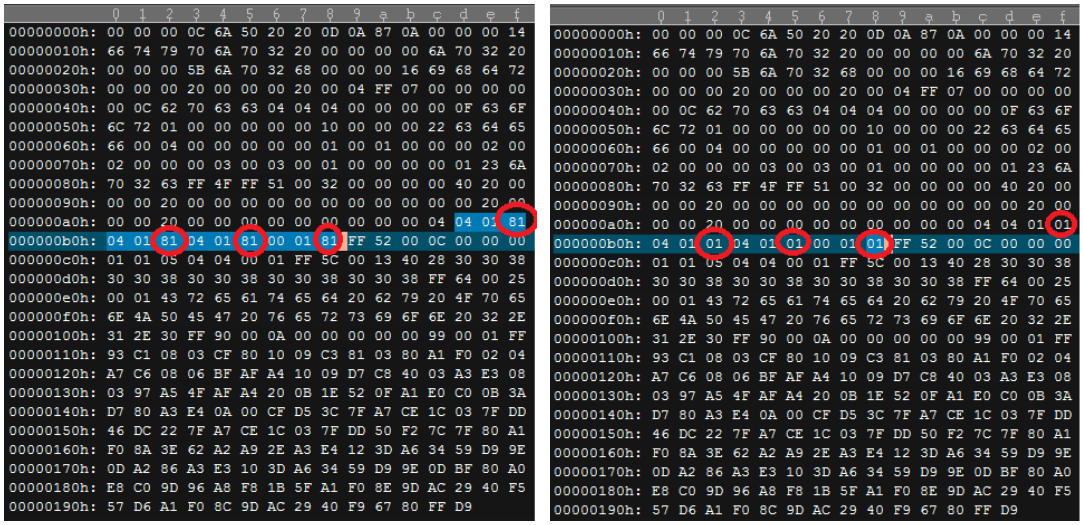
Old value = 129 '\201'
New value = 0 '\000'
__mempcpy_ia32 () at ../sysdeps/i386/i686/multiarch/../mempcpy.S:54
54      ../sysdeps/i386/i686/multiarch/../mempcpy.S: No such file or directory.
(rr) bt
#0  __mempcpy_ia32 () at ../sysdeps/i386/i686/multiarch/../mempcpy.S:54
#1  0xb758e255 in __GI__IO_file_xsgetn (fp=0x84d4020, data=0xb7300008, n=1048576) at fileops.c:1392
#2  0xb759022e in __GI__IO_sgetn (fp=0x84d4020, data=0xb7300008, n=1048576) at genops.c:467
#3  0xb75837d7 in __GI__IO_fread (buf=0xb7300008, size=1, count=1048576, fp=0x84d4020) at iofread.c:38
#4  0xb776acbb in opj_read_from_file (p_buffer=0xb7300008, p_nb_bytes=1048576, p_file=0x84d4020)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/openjpeg.c:93
#5  0xb774a5e7 in opj_stream_read_data (p_stream=0x84d4180, p_buffer=0xbf8f4324 "", p_size=8, p_event_mgr=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/cio.c:309
#6  0xb7763cd5 in opj_jp2_read_boxhdr (box=0xbf8f4390, p_number_bytes_read=0xbf8f4374, cio=0x84d4180, p_manager=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/jp2.c:475
#7  0xb7767e99 in opj_jp2_read_header_procedure (jp2=0x84d5228, stream=0x84d4180, p_manager=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/jp2.c:2128
#8  0xb7767fc8 in opj_jp2_exec (jp2=0x84d5228, p_procedure_list=0x84d74b8, stream=0x84d4180, p_manager=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/jp2.c:2260
#9  0xb7768c31 in opj_jp2_read_header (p_stream=0x84d4180, jp2=0x84d5228, p_image=0xbf8f4474, p_manager=0x84d51fc)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/jp2.c:2662
#10 0xb776b18f in opj_read_header (p_stream=0x84d4180, p_codec=0x84d51d0, p_image=0xbf8f4474)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/openjpeg.c:391
#11 0x0804c038 in main (argc=5, argv=0xbf8f9604) at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/bin/jp2/opj_decompress.c:1312
(rr) break /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/openjpeg.c:93
Breakpoint 17 at 0xb776aca9: file /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/openjpeg.c, line 93.
(rr) reverse-continue
Continuing.

Breakpoint 17, opj_read_from_file (p_buffer=0xb7300008, p_nb_bytes=1048576, p_file=0x84d4020)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/lib/openjp2/openjpeg.c:93
93              OPJ_SIZE_T l_nb_read = fread(p_buffer,1,p_nb_bytes,p_file);
(rr)
```

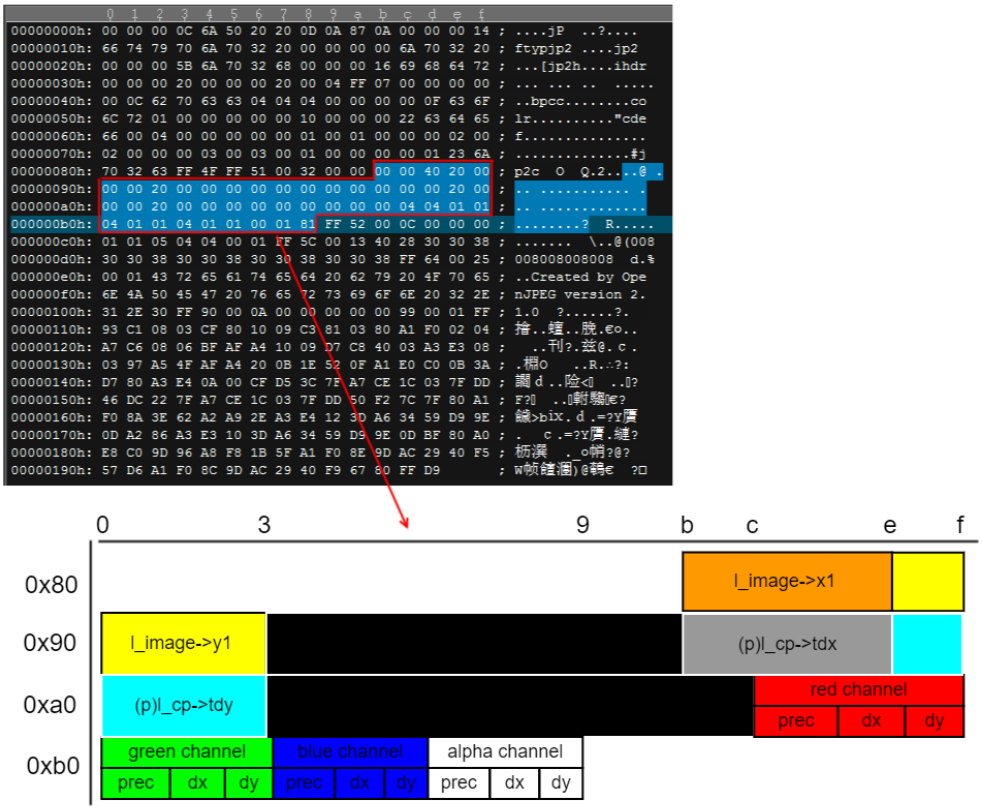
Finally, I find the value is directly read from the input file i.e. PoC. The position is shown as the following:



If I modify the value at **offset 0xb8** to be **0x01** shown as the following (**right**), I find the modified input image will not cause crash any more, which can be viewed as one verifying method. Also, if I modify the values at **offset 0xaf (i.e. comps[0].dy), offset 0xb2 (i.e. comps[1].dy) and offset 0xb5 (i.e. comps[2].dy)** to be **0x81** shown as the following (**left**), the crash will not happen and "opj_decompress" finishes normally.



The crash-related buffers' allocation and access depends on values from offset 0x8B to offset 0xB8 of the input PoC file, which is shown as the following:



X1 (4 bytes): value of l_image->x1

Y1 (4 bytes): value of l_image->y1, l_cp->tdx, l_cp->tdy, p_cp->tdx and p_cp->tdy

Red channel (3 bytes): parameters of red channel, including prec, dx and dy, i.e. comps[0].prec, comps[0].dx and comps[0].dy

Green channel (3 bytes): parameters of green channel, including prec, dx and dy, i.e. comps[1].prec, comps[1].dx and comps[1].dy

Blue channel (3 bytes): parameters of blue channel, including prec, dx and dy, i.e. comps[2].prec, comps[2].dx and comps[2].dy

Alpha channel (3 bytes): parameters of alpha channel, including prec, dx and dy, i.e. comps[3].prec, comps[3].dx and comps[3].dy

The computations of parameters related with buffer allocations and buffer accesses are:

$$\text{comps[i].h} = \frac{l_comp_y1 - l_comp_y0}{2^{comps[i].factor}}$$

$$\text{l_comp_y1} = \frac{l_y1 + comps[i].dy - 1}{comps[i].dy}$$

$$\text{l_comp_y0} = \frac{l_y0 + comps[i].dy - 1}{comps[i].dy}$$

$$\text{l_y1} = \max\{\text{l_image} \rightarrow \text{y1}, (\text{p_cp} \rightarrow \text{ty0} + (\text{p_cp} \rightarrow \text{th} - 1) * \text{p_cp} \rightarrow \text{tdy}) + \text{p_cp} \rightarrow \text{tdy}\}$$

$$\text{l_y0} = \max\{p_cp \rightarrow ty0, \quad l_image \rightarrow y0\}$$

$$\text{p}_{\text{cp}} \rightarrow \text{th} = \frac{l_{image} \rightarrow y1 - l_{cp} \rightarrow ty0 + l_{cp} \rightarrow tdy - 1}{l_{cp} \rightarrow tdy}$$

$$0 \leq \text{i} < 4$$

$$\text{comps[i].w} = \frac{l_comp_x1 - l_comps_x0}{2^{comps[i].factor}}$$

$$\text{l_comp_x1} = \frac{l_x1 + comps[i].dx - 1}{comps[i].dx}$$

$$\text{l_comp_x0} = \frac{l_x0 + comps[i].dx - 1}{comps[i].dx}$$

$$\text{l_x1} = \max\{\text{l_image} \rightarrow \text{x1}, (\text{p_cp} \rightarrow \text{tx0} + (\text{p_cp} \rightarrow \text{tw} - 1) * \text{p_cp} \rightarrow \text{tdx}) + \text{p_cp} \rightarrow \text{tdx}\}$$

$$\text{l_x0} = \max\{p_cp \rightarrow tx0, \quad l_image \rightarrow x0\}$$

$$\text{p_cp} \rightarrow \text{tw} = \frac{l_image \rightarrow x1 - l_cp \rightarrow tx0 + l_cp \rightarrow tdx - 1}{l_cp \rightarrow tdx}$$

$$0 \leq \text{i} < 4$$

(Note: in the debugging, **p_cp->tx0**, **p_cp->ty0**, **l_image->x0** and **l_image->y0** are the default values after allocation, i.e. 0).

```
1771        if ((force_split == 0) &&
1772                (ncomp == 2 /* GRAYA */
1773              || (ncomp > 2 /* RGB, RGBA */
1774                  && image->comps[0].dx == image->comps[1].dx
1775                  && image->comps[1].dx == image->comps[2].dx
1776                  && image->comps[0].dy == image->comps[1].dy
1777                  && image->comps[1].dy == image->comps[2].dy
1778                  && image->comps[0].prec == image->comps[1].prec
1779                  && image->comps[1].prec == image->comps[2].prec
1780                  )))
```

Code in line 1771~1780 :convert.c makes sure:

$$\text{comps[0].dx} = \text{comps[1].dx} = \text{comps[2].dx}$$

$$\text{comps[0].dy} = \text{comps[1].dy} = \text{comps[2].}dy$$

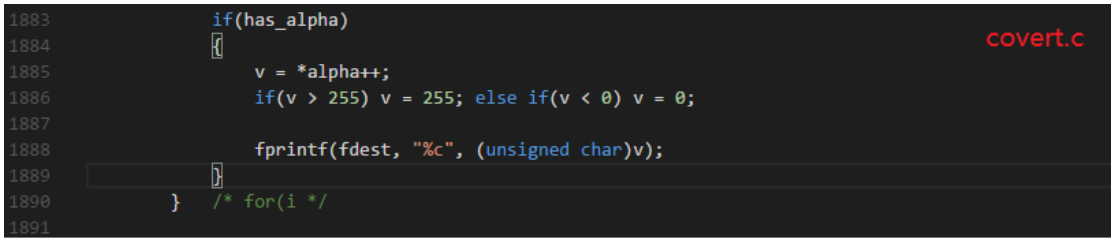
$$\text{comps[0].prec} = \text{comps[1].prec} = \text{comps[2].prec}$$

Next, I know

RED GREEN BLUE: $\text{len(comps[0].data)} = len(comps[1].data) = len(comps[2].data) = comps[0].w * comps[0].h * sizeof(opj_uint32)$

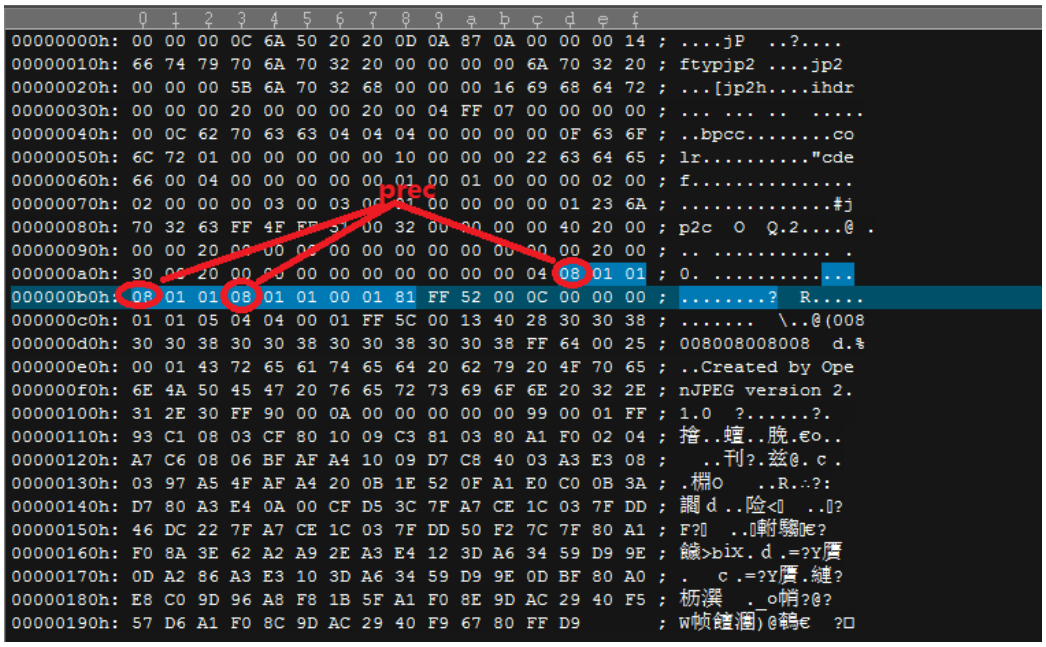
ALPHA: $\text{len(comps[3].data)} = \text{comps[3].w} * \text{comps[3].h} * \text{sizeof(opj_uint32)}$

So, I can control the total loop times **wr*hr** by crafting **comps[0].w** and **comps[0].h** which can be achieved by crafting the corresponding fields of input image file, e.g. **X1**, **Y1**, **dx** (red & green & blue channel), **dy**(red & green & blue channel) (**note: make sure that dx(dy) of red, green and blue channel are the same**). I can control length of **comps[3].data** (alpha channel) by crafting **comps[3].w** and **comps[3].h** which can be achieved by crafting the corresponding fields of input image file, including **X1**, **Y1**, **comps[3].dx** (alpha channel) and **comps[3].dy** (alpha channel). That's to say, I can control the total loop times and length of the buffer to be accessed, i.e. I can control the size of some heap chunk and the heap chunk's beyond-access size.

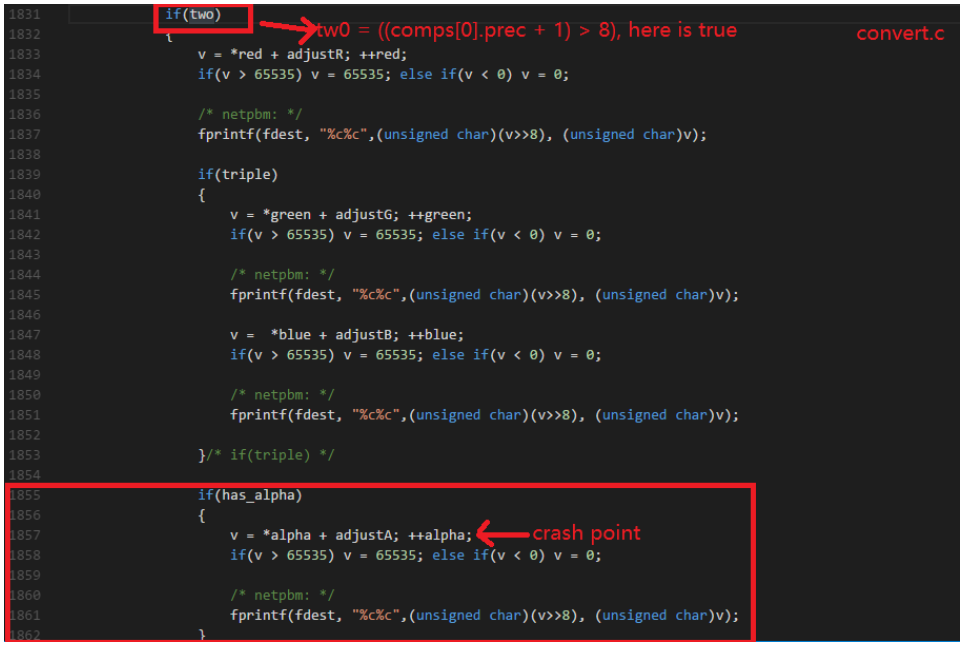


From the above picture, the read out alpha values are written out to a destination file; when out-of-bound read happening, some extra information can be leaked out; the vulnerability can cause information disclosure.

Besides, crash can be trigged at other code line, e.g. make **comps[0\1\2].prec** greater than **0x07** and less than **0x11**, like the following:



The crash will happen at line 1857 :convert.c:




```
[INFO] Stream reached its end !

Program received signal SIGSEGV, Segmentation fault.
0x080517c3 in imagetopnm (image=0x8063890, outfile=0xbfffcf44 "a.ppm",
    force_split=0)
    at /home/fire/bing/afl/libraries/openjpeg-2.1.2/src/bin/jp2/convert.c:1857
1857                        v = *alpha + adjustA; ++alpha;
(gdb)
```

Besides, comps[1].data (green buffer) and comps[2].data (blue buffer) will not overflow due to the restrictions: $\mathrm{comps}[0].\mathrm{dx}(\mathrm{dy}) = comps[1].dx(dy) = comps[2].dx(dy)$

Conclusion

The vulnerability exists in openjpeg-2.1.2 (particularly, decompress parts). The vulnerability is a heap buffer overflow vulnerability, which can cause out-of-bound read. The vulnerability is caused by some lacked check: the code make sure parameters (dx, dy) of red channel, green channel and blue channel are the same, however, does not do it for alpha channel's parameters (dx, dy). The vulnerability can cause Denial-of-Service and Information Disclosure. I am not sure whether it can cause Remote-Code-Execution.

Author

Name: Bingchang, Liu @VARAS of IIE
Org: IIE (http:// iie.ac.cn)